

Sección 3 — Gramática

Documentación técnica: ejercicios cloze con spaCy, distractores híbridos paradigma/coseno y filtros de unicidad de respuesta

1. Qué resuelve esta sección

La Sección 3 convierte los textos en alemán de la plataforma en **ejercicios de gramática**. La organización es **por tema**: primero se presenta la regla (lección con resumen y tabla de referencia) y a partir de ella se practica con ejercicios **cloze** de opción múltiple: una frase real del corpus con un hueco y 4 opciones (la respuesta + 3 distractores). Cuatro temas iniciales: declinación del artículo, preposición y caso, conjugación del verbo y verbos separables.

Todo el trabajo lingüístico (localizar el hueco, construir y ordenar los distractores) ocurre **offline** en el pipeline Python con spaCy; el frontend solo baraja y presenta. El reto central no es encontrar huecos sino garantizar que el ejercicio tenga **respuesta única**: un distractor que también produzca una frase válida no evalúa nada. Cada tema lleva su propio criterio formal de exclusión (sección 6).

2. Mapa de archivos

Pieza	Archivo	Qué contiene
Generador de ejercicios	pipeline/gramatica.py	spaCy de_core_news_md: detección de huecos por tema, distractores híbridos, filtros de unicidad, selección estratificada
Datos	src/data/gramatica.json	temas[] (lección: resumen + tabla) y ejercicios{tema: [...]}; lo sobrescribe el pipeline
Motor puro	src/engine/gramatica.js (+ .test)	Sesión y opciones deterministas por semilla; corrección y resumen
UI selector de temas	src/secciones/gramatica/Gramatica.jsx	Ruta /gramatica: tarjetas de tema con lección y conteo
UI sesión de práctica	src/secciones/gramatica/Ejercicios.jsx	Ruta /gramatica/:tema: regla desplegable, hueco, opciones, feedback
Estilos	src/secciones/gramatica/gramatica.css	Reutiliza el layout de lectura.css y la paleta de repaso.css

3. Formato de datos

Un ejercicio parte la frase en tres trozos alrededor del hueco, así la UI lo renderiza sin volver a tokenizar. La pista explica la regla que decide la respuesta y la fuente identifica la lectura de origen (nivel · título).

```
{ "id": "declinacion-03",
  "antes": "Sie fährt mit ",
  "respuesta": "dem",
  "despues": " Fahrrad zur Schule.",
  "distractores": ["den", "das", "der"],
  "pista": "Dativo · neutro · singular.",
```

```
"fuente": "principiante · Un día de Ana",
"nivel": "principiante" }
```

Cada tema lleva además su **lección**: { id, titulo, nivel, resumen, tabla: { cabecera, filas } }. La UI la muestra en la portada del tema y como desplegable («Ver la regla») durante la práctica. El nivel del tema clasifica su dificultad (decisión del usuario: declinación, conjugación y separables = principiante; preposición y caso = intermedio); el del ejercicio es el de su lectura de origen. La navegación es **por lectura**: el motor reagrupa el JSON con agruparPorLectura (lecturas ordenadas por nivel ascendente; dentro de cada una, los ejercicios agrupados por tema en orden de dificultad del tema).

4. Detección de huecos (spaCy)

El pipeline recorre las frases en alemán de **todas** las lecturas (solo frases de hasta 18 tokens: las de los libros llegan a 40+ y no funcionan como cloze). Cada tema define qué token se convierte en hueco:

Tema	Criterio sobre el análisis de spaCy
Declinación	pos=DET con lemma «der» (artículo definido); la pista sale de la morfología Case/Gender/Number del token
Preposición y caso	pos=ADP presente en el inventario dativo/acusativo, y con marca de caso visible en su sintagma (ver §6)
Conjugación	pos=VERB/AUX con VerbForm=Fin; la pista sale de Person/Number
Separables	dep=svp (prefijo verbal separado); el lema del verbo se reconstruye prefijo + lema del head (vor + bereiten → vorbereiten), igual que en la Sección 1

5. Distractores híbridos: paradigma + coseno

Un método único no basta. La **similitud coseno** a secas produce distractores flojos en clases cerradas (los vecinos vectoriales de «der» no son necesariamente artículos); el **paradigma** a secas no sabe cuáles de las formas rivales son plausibles en el contexto. El híbrido asigna a cada método el papel que hace bien:

1. El **paradigma morfológico define el conjunto** de candidatos: las otras 5 formas del artículo; las preposiciones del caso opuesto; las demás formas del mismo lema verbal atestiguadas en el corpus (índice lema → formas invertido del léxico de la Sección 1); los demás prefijos separables.
2. La **similitud coseno ordena el conjunto**: con los vectores de spaCy (de_core_news_md, 20.000 × 300), se puntúa $\cos(v_{\text{respuesta}}, v_{\text{candidato}})$ y se toman los k=3 más parecidos — los *hard negatives*, los más difíciles de descartar para el alumno.

$\cos(u, v) = (u \cdot v) / (||u|| \cdot ||v||)$ empate → orden alfabético (determinista)

6. Filtros de unicidad de respuesta

Ordenar por coseno maximiza la dificultad, pero un distractor «demasiado bueno» puede ser directamente **válido**. Cada tema excluye sus alternativas válidas con un criterio verificable:

Tema	Criterio de exclusión	Ejemplo que evita
Declinación	Las demás formas del paradigma violan caso/género/número por construcción (el sustantivo fija género y número; la sintaxis, el caso)	—
Preposición	Pool = preposiciones del caso OPUESTO; además el hueco solo se genera si el sintagma lleva marca de caso visible (DET/ADJ/PRON declinado)	«um sieben Uhr»: sin artículo, «nach sieben Uhr» también sería válida
Conjugación	Re-parseo con sustitución: el candidato solo vale si en contexto NO es forma finita o su persona/número no concuerdan con el sujeto	«ging» → «geht»: otro tiempo, misma concordancia → frase válida → excluido

Tema	Criterio de exclusión	Ejemplo que evita
Separables	La respuesta debe formar un verbo atestiguado en el léxico del corpus (descarta svp espurios del parser) y los distractores deben NO formarlo	aufmachen vs zumachen: ambos válidos → «zu» no puede ser distractor de «auf»

El filtro de conjugación es el más caro: por cada candidato se re-parsea la frase con la sustitución hecha y se lee la morfología del token resultante. Para acotarlo, se valida en orden de coseno descendente y se corta al llegar a k distractores.

7. Selección estratificada

Los candidatos se acumulan por (tema, fuente) con una cota de 12 por fuente, y la selección final hace **round-robin** entre fuentes ordenadas por nivel (principiante → intermedio → avanzado) hasta el tope de 40 por tema. Sin esto, el tope se llenaba con la primera lectura en orden alfabético (Immensee) y el resto del corpus no aportaba nada; con esto, cada tema mezcla los tres niveles y 9–12 fuentes distintas.

8. Motor puro (JS) y UI

8.1 src/engine/gramatica.js

Funciones puras testeadas con Vitest: `opcionesDe(ejercicio, semilla)` baraja respuesta+distractores de forma determinista (mismo orden entre renders, distinto entre ejercicios); `agruparPorLectura(data)` reagrupa los ejercicios por lectura de origen (orden: nivel ascendente; dentro, por tema); `claveEjercicio / lecturaCompletada` gobiernan el progreso (clave estable fuente|antes|respuesta: sobrevive a regenerar los ids); `esCorrecta` y `resumenSesion` completan la sesión. El azar entra siempre por el LCG determinista de `board.js`, con una corrección: ese generador puede emitir valores fuera de [0,1) (hash de semilla negativo), así que el motor toma la parte fraccionaria antes de barajar.

8.2 Flujo de la UI

/gramatica lista las **lecturas** del corpus con ejercicios, ordenadas de principiante a avanzado, cada una con su etiqueta de nivel, el conteo y una palomita ✓ si ya está completada. **/gramatica:lectura** recorre TODOS los ejercicios de esa lectura agrupados por tema (los temas de principiante primero); cada ejercicio muestra la regla de su tema en un desplegable, la frase con hueco y 4 opciones; al responder se colorea la elección y la correcta y se muestra la pista. Cada acierto se persiste (`gramatica.hechos.v1`); la lectura queda completada cuando todos sus ejercicios se han respondido bien alguna vez. `gramatica.json` se carga por **dynamic import**: chunk aparte, fuera del bundle inicial, como el léxico.

9. Decisiones y trampas

Tema	Decisión / trampa
Respuesta única	Es el requisito central del diseño; cada tema tiene su filtro formal (§6). Un cloze con dos respuestas válidas no evalúa nada.
LCG de board.js	<code>state/(m-1)</code> con hash negativo produce valores negativos; el motor de gramática normaliza a [0,1) sin tocar <code>board.js</code> (cambiaría los tableros del juego)
svp espurios	El parser etiqueta svp cosas que no son prefijos separables («gab die Hand darauf» → *daraufgeben); la atestiguación en el léxico los descarta
Separables ≠ gramática pura	Elegir el prefijo es en parte semántico (la pista da el verbo); la atestiguación garantiza que los distractores no formen verbo real
Regeneración	<code>gramatica.json</code> lo sobrescribe el pipeline; no editar a mano. El proceso es determinista: mismo corpus → mismo JSON
Python + umlauts	Siempre <code>PYTHONUTF8=1</code> y <code>ensure_ascii=False</code> (regla del repo)

10. Cómo regenerar cada artefacto

```
python pipeline/gramatica.py          # src/data/gramatica.json (los 4 temas)
python pipeline/gramatica.py conjugacion # un solo tema
npm test                             # tests del engine
python docs/generar_doc_seccion3.py    # este PDF
python docs/generar_metricas_seccion3.py
python docs/generar_autoaprendizaje_seccion3.py
```